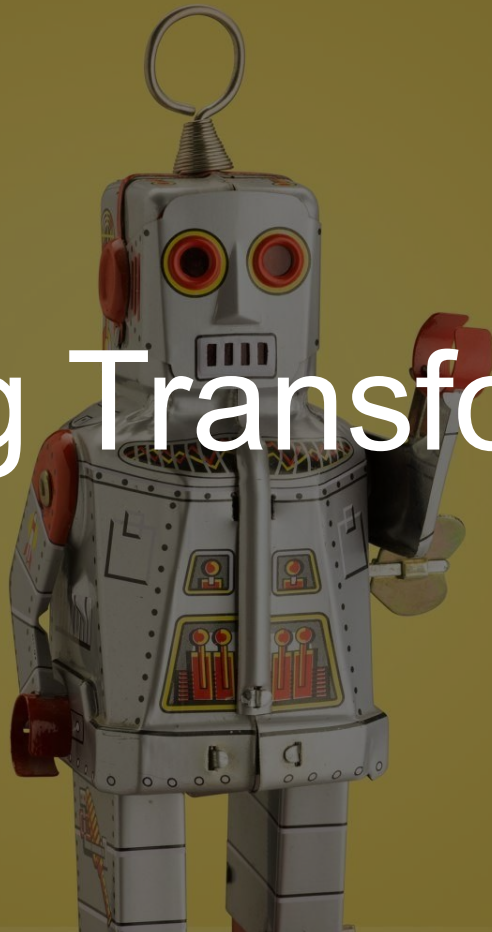
Newton School of Technology

# Join the lecture online on your dashboard.

# Training Transformers



# Topics in this lecture

- Flash Attention
- Supervised Fine Tuining
- Low Rank Adapter (LoRA)
- Knowledge Distillation

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

# IV: Flash Attention

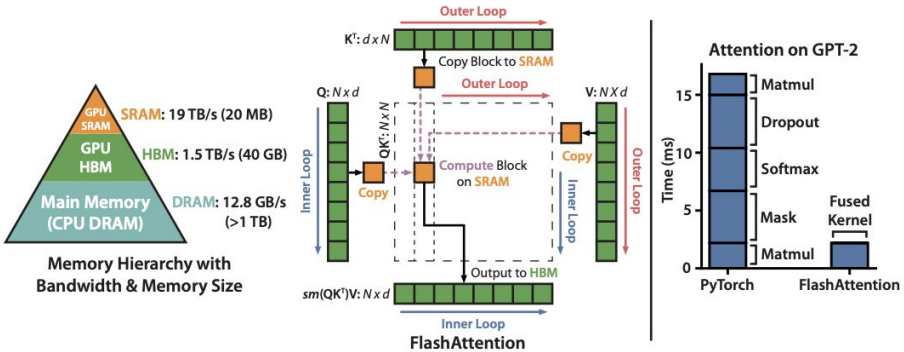
# FLASHATTENTION: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University
[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY

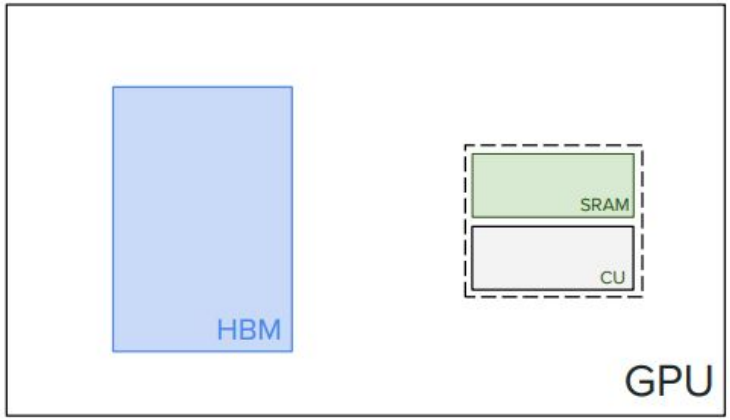
{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

2022



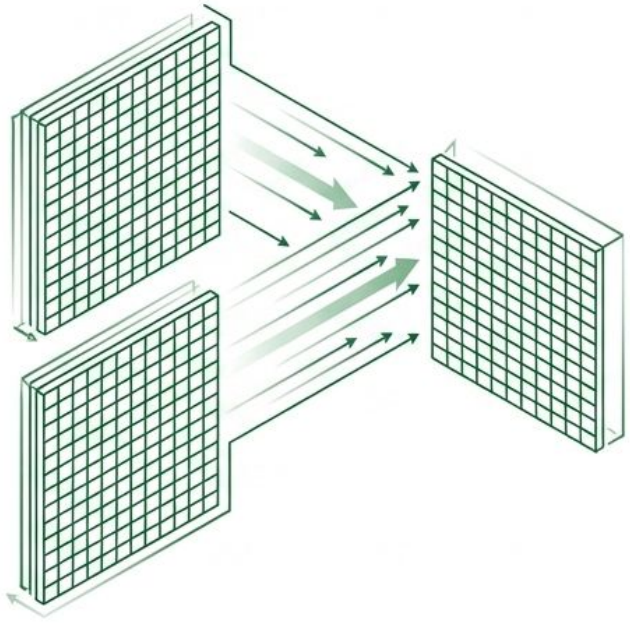
# Optimization by Flash Attention

- Flash attention leverages the internal structure of the GPU to speed of computation of Attention.
- Computation @ PetaFLOP speed does not matter, if there is no data to be processed.
- Operations are of two types:
  - Compute-bound (e.g. matrix multiplication)
  - Memory-bound (involving frequent read-writes, like softmax, element-wise operations, etc.)



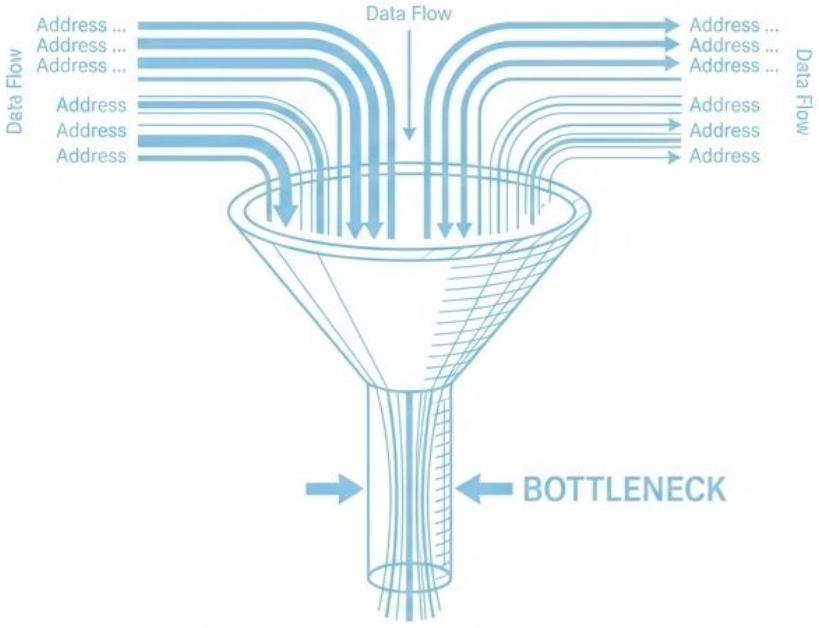
## Compute-Bound Operations

### Matrix Multiplication



## Memory-Bound Operations

### Softmax, Element-wise operations

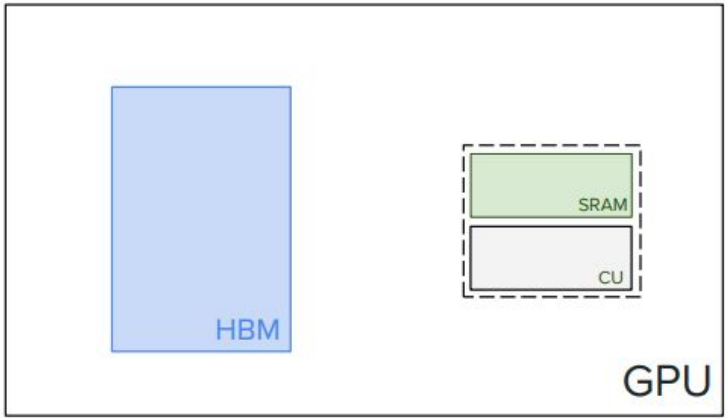


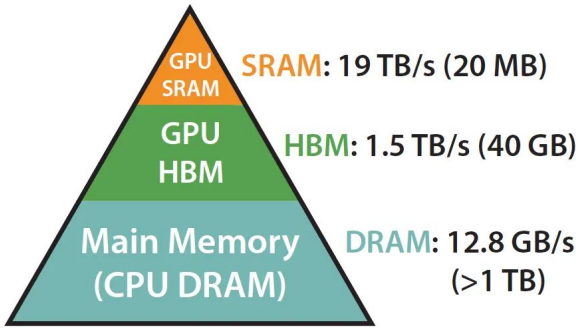

Computation at PetaFLOP speed is meaningless if the GPU is starved for data.
Frequency of read-writes limits performance, not sheer mathematical capability.

# Optimization by Flash Attention

- A GPU consists of:
  - one HBM (High Bandwidth Memory) &
  - Multiple [CU+SRAM].

- HBM → Big and Slow
- SRAM → Small and very fast

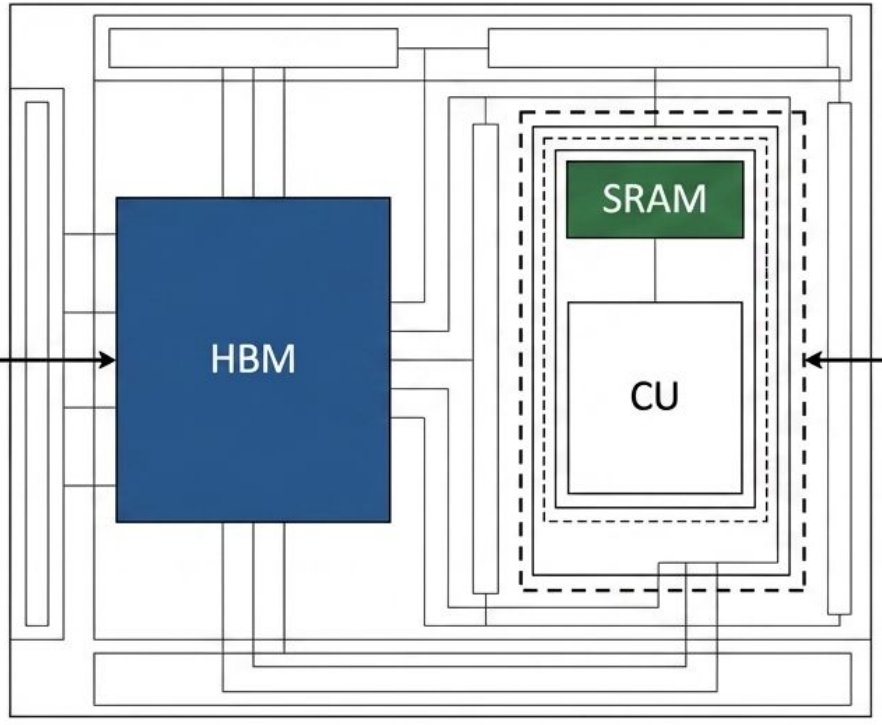



Memory Hierarchy with Bandwidth & Memory Size

# GPU Architecture Fundamentals: The Hardware Canvas

HBM (High Bandwidth Memory): Centralized, Big, and Slow.




Multiple [CU + SRAM] Units: Distributed, Small, and Very Fast.

**Key Hardware Insight:** The physical distance and bandwidth limit between the central HBM and the distributed Compute Units is the direct cause of the latency penalty.

# How does the Standard Attention algorithm interact with this memory architecture?

# Algorithm 0: The Standard Attention Baseline

Standard implementations require matrices $Q$, $K$, $V$ to reside fully in the High Bandwidth Memory (HBM).

1. Compute Similarity Scores

$$S = QK^T$$

2. Apply Activation

$$P = softmax(S)$$

3. Compute Output

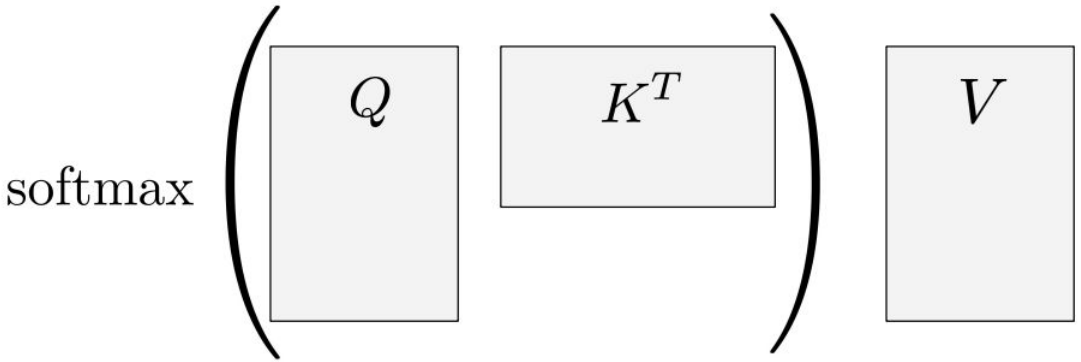
$$O = PV$$

# Standard Attention

**Algorithm 0** Standard Attention Implementation

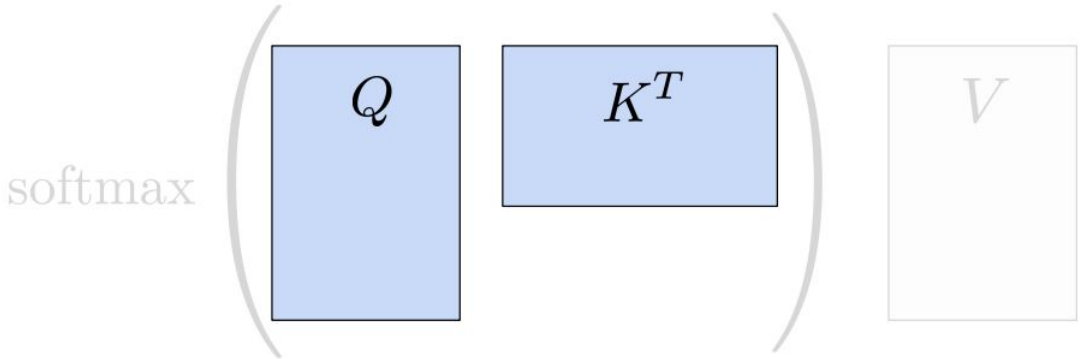
**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.



# Standard Attention

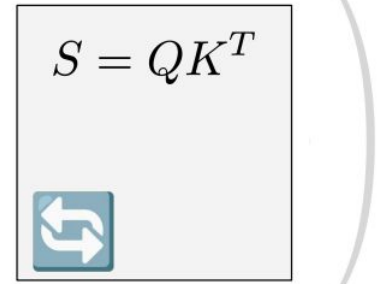
- Load Q, K from HBM by blocks.



# Standard Attention

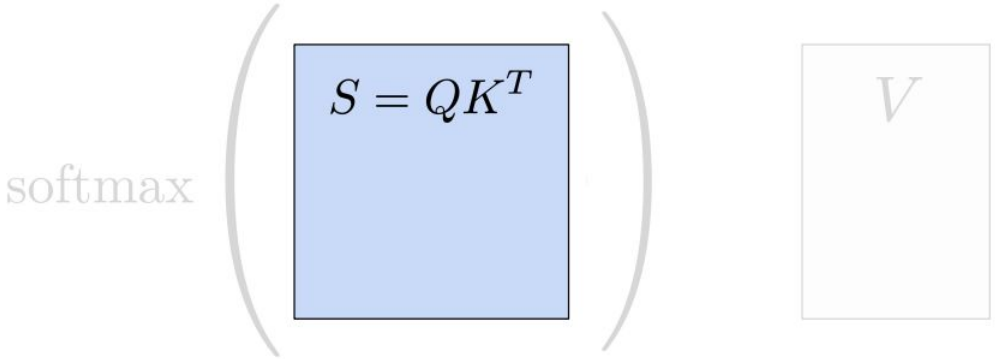
- Compute S.

$$\text{softmax}\left( S = QK^T \right) V$$



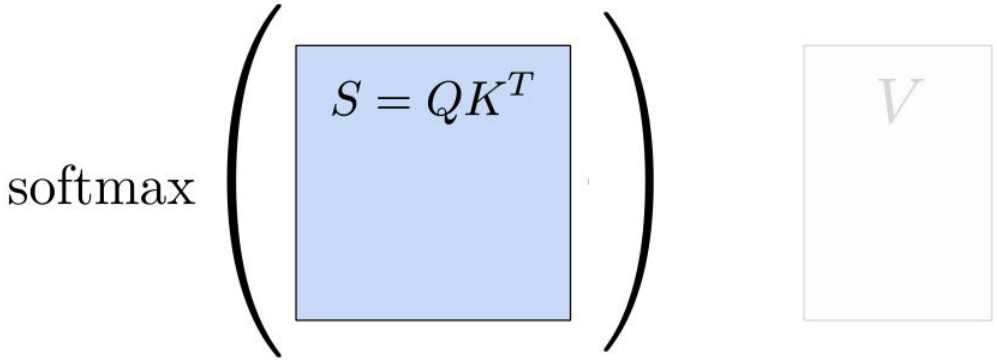
# Standard Attention

- Write S to HBM.



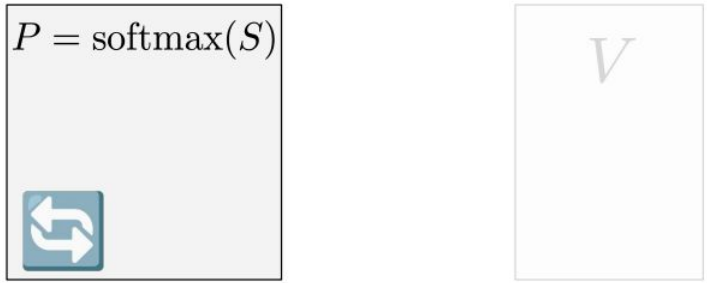
# Standard Attention

- Read S from HBM.



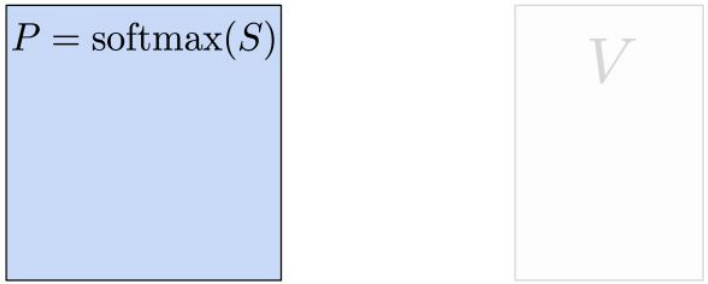
# Standard Attention

- Compute P.



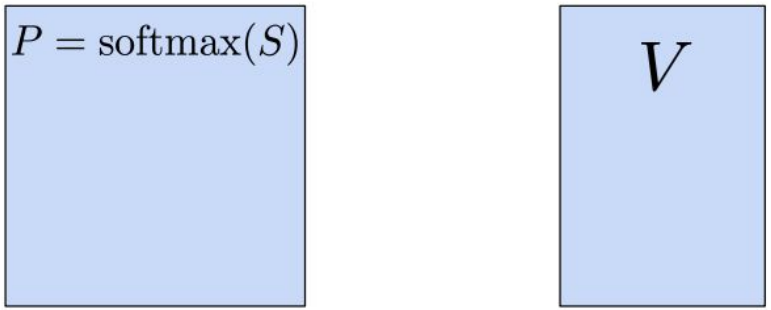
# Standard Attention

- Write P to HBM.



# Standard Attention

- Load P, V from HBM.

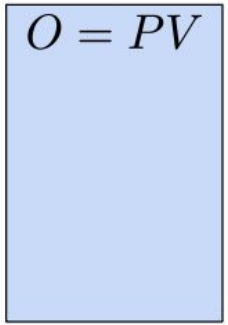


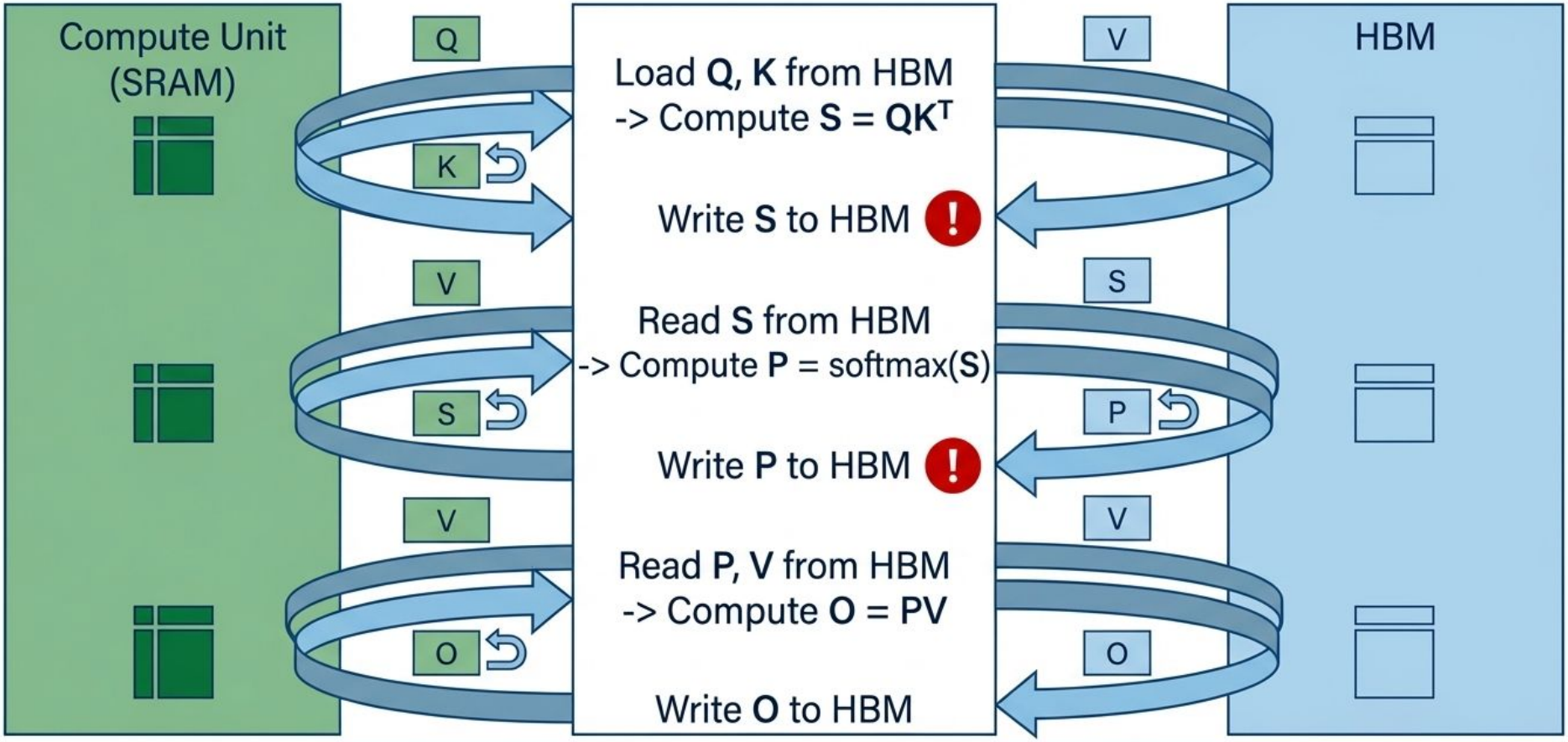
# Standard Attention

- Compute O.

# Standard Attention

- Write O to HBM.






Standard Attention demands redundant read/writes of massive intermediate matrices (S and P) to the slow HBM. This is the ultimate **bottleneck.**

# Diagnosing the Structural Redundancy

## Point 1: Massive Data Movement

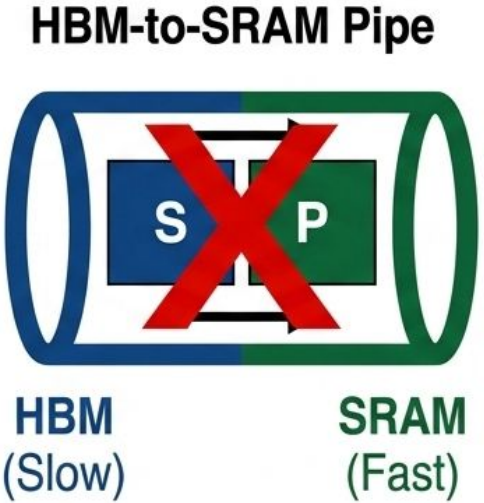
Algorithm 0 forces continuous read/write cycles of large N × N intermediate matrices (S and P) across the slow memory bridge.

## Point 2: Memory-Bound Execution

The high-speed compute units (SRAM) sit completely idle while waiting for data to travel from the HBM. PetaFLOP potential is fundamentally wasted.

## Point 3: The Engineering Imperative

We must completely remove redundant HBM read-writes from the execution pipeline.

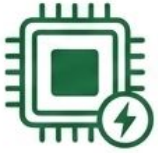


# If redundant HBM read-writes are the fatal bottleneck, how do we compute Attention without ever leaving the SRAM?

# The Flash Attention Paradigm Shift

## Compute in Blocks

Process attention dynamically in small tiles rather than relying on full-scale matrices.

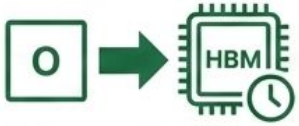


## Keep in SRAM

Trap intermediate data entirely within the high-speed cache limits.

## Fuse Operations

Combine matrix multiplication and softmax math into a single, contiguous flow.




## Write Once

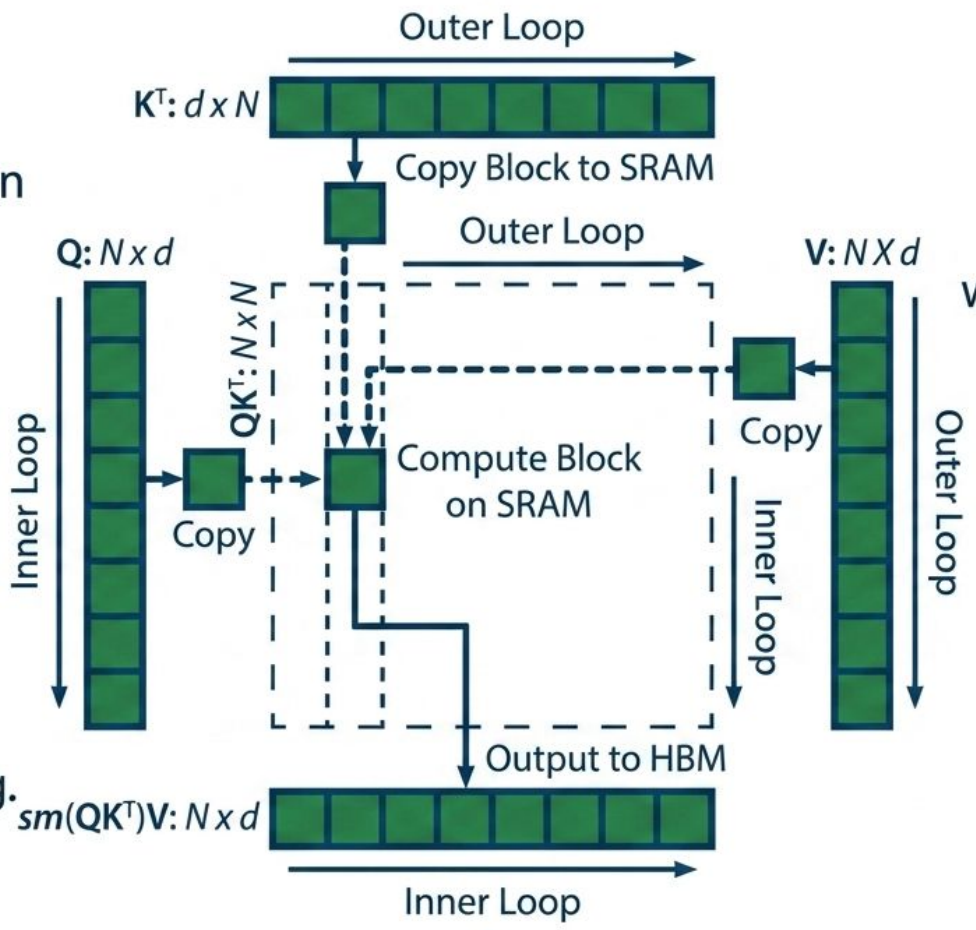
Send only the final, complete output matrix (O) back across the slow bridge to HBM.

## 1. Tiling

Compute attention in small, manageable blocks.

## 2. Locality

Keep all active computation strictly within the fast SRAM.




## 3. Fusion

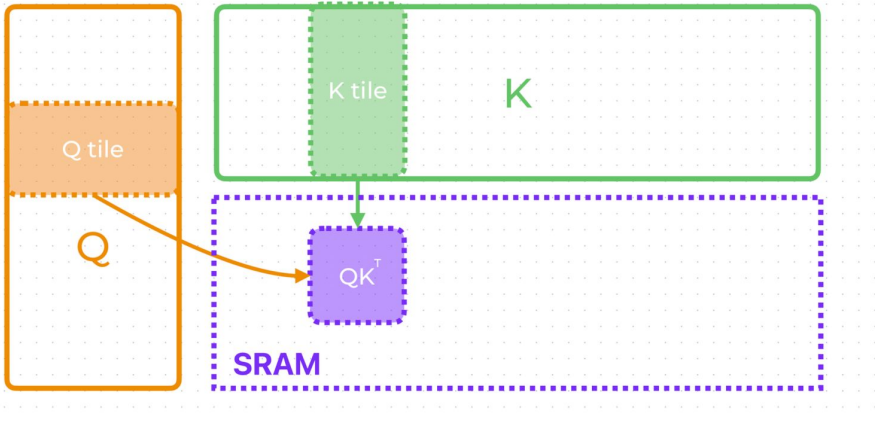
Fuse operations together to avoid intermediate staging.

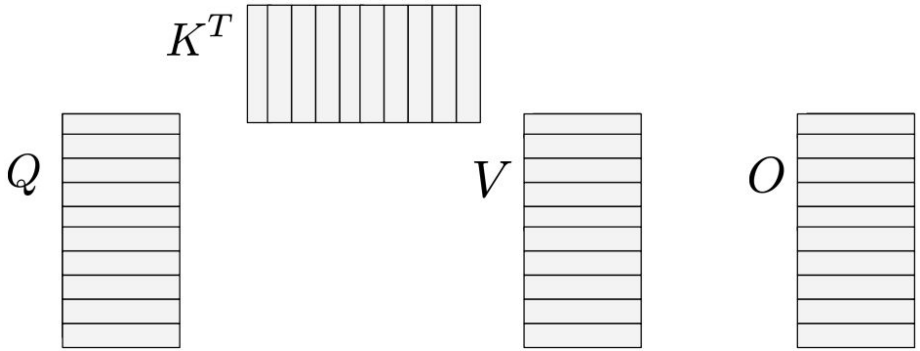
## 4. Finality

Only write the final output (O) back to the HBM.

# Flash attention in action

Minimize read/writes to HBM using “tiling” via SRAM.



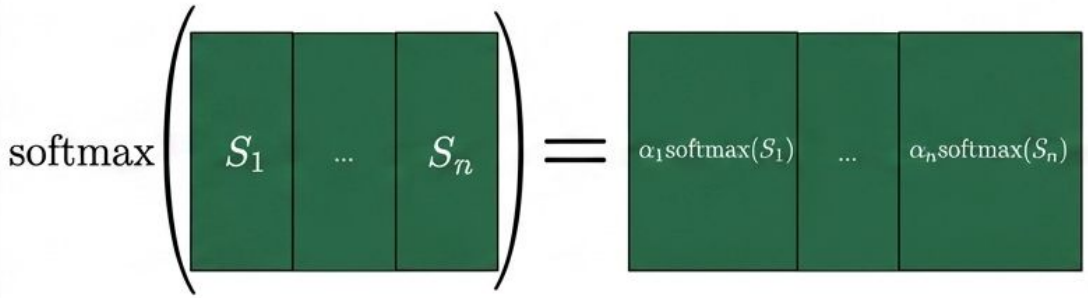
**Softmax inherently requires an entire row of data to calculate accurately. How can we possibly process it in isolated, fragmented blocks?**

# The Algorithmic Challenge: Concatenated Softmax

## The Problem

Standard Softmax requires the sum of all elements in a full row.

How do we compute an accurate global activation when we only hold a tiny local block?




## The Solution

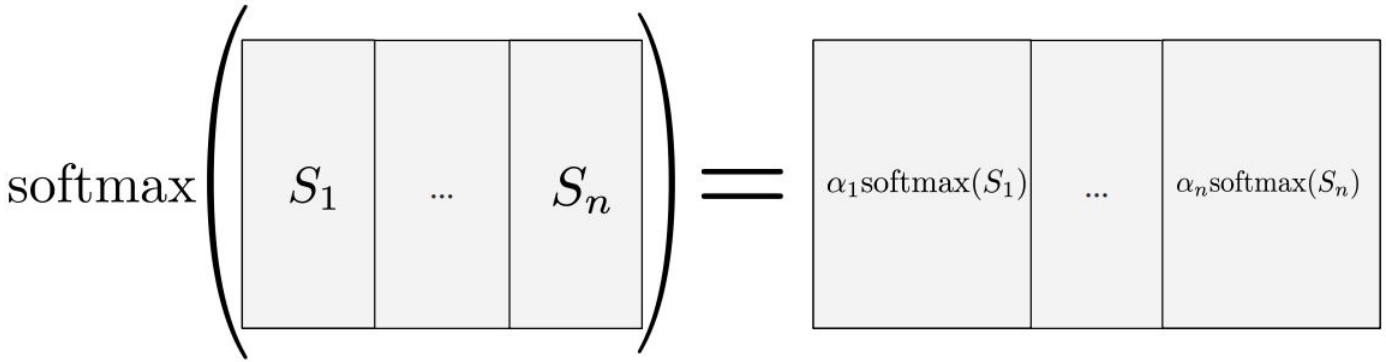
The Concatenated Softmax Algorithm:

Process each block $S_i$ independently inside the SRAM cache.

Track the local maximums dynamically, and later combine them using a running correction factor ($\alpha_i$).

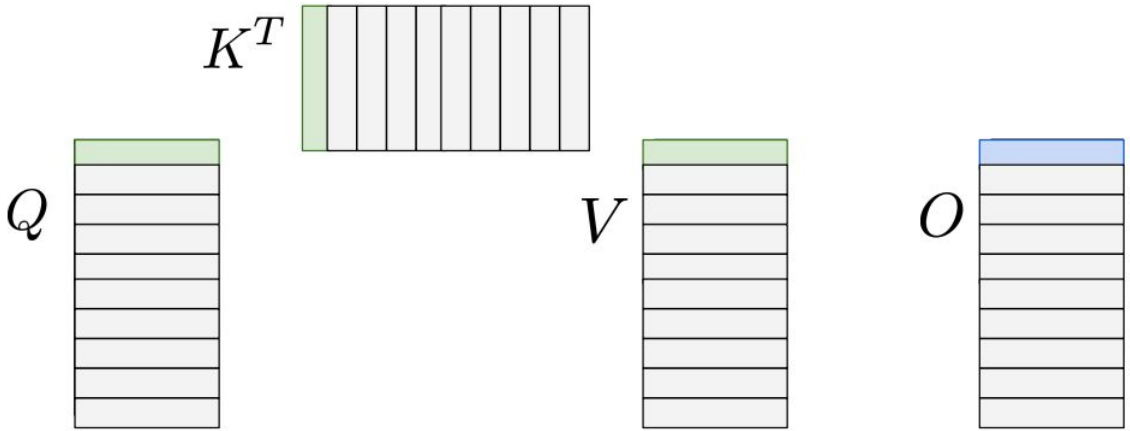
# Flash attention in action

- No need to compute the full $S = QK^T$ before applying softmax.
- Split $S$ into $S_1, S_2, \ldots, S_n$.
- Each block $S_i$ fits in SRAM.
- Process each block independently.
- Then combine using the correction factor $\alpha_i$ to get final result. (Concatenated Softmax Algorithm)



# Flash attention in action

- Load blocks of Q, K, V to SRAM.
- Compute block of O.
- Write the result to HBM.



# Flash attention in backward pass

- Standard Attention (backward pass)
- Forward pass stores:
  - $S = QK^T$
  - $P = softmax(S)$

- Backward pass:
  - Read S, P from HBM.

# Flash attention in backward pass

- **Flash attention strategy**
- Do not store S, P.
- Instead recompute them using tiling in SRAM.

- Flash attention trades extra computation for reduced memory access.

# The Backward Pass Dilemma & Brilliant Trade-off

Training gradients normally require the intermediate S and P matrices computed during the forward pass.

| Standard Approach | Flash Attention Approach |
| --- | --- |
| Read the stored S and P directly from the HBM. | Do not store S and P. Recompute them completely from scratch on the fly during the backward pass using SRAM tiling. |
|  |  |
| ⚠ Fast compute, catastrophic memory latency | ✅ Extra compute cost, zero memory latency |

To save memory bandwidth across the hardware chasm, the algorithm actively chooses to do the complex math twice.

**By deliberately forcing the system to recompute data from scratch, aren't we significantly  increaasing the total mathematical workload? Why is this a victory?**

| | Standard Attention | Flash Attention |
|---|---|---|
| Memory Access (The Bottleneck) | Maximum HBM read/writes (Extremely Slow) | Minimal HBM access; strict SRAM utilization (Extremely Fast) |
| Computational Volume (The Workload) | Standard FLOPs | Increased FLOPs (due to recomputation) |

## Trading Compute for Time

Flash Attention makes a strategic, asymmetric trade. It intentionally increases the volume of cheap, fast computation (FLOPs) to drastically eradicate the volume of expensive, slow memory access (HBM I/O). The result is mathematically exact Attention computed significantly faster.

# Flash Attention trades extra computation for drastically reduced memory access.

| The Limitation | The Algorithmic Shift | The Reality |
| --- | --- | --- |
| Physical GPU compute units are fundamentally, infinitely faster than memory bandwidth. | Tiling and operation fusion eliminate the need to ever read or write N x N intermediate matrices. | Recomputing data locally inside the SRAM cache is significantly faster than reading stored data from the HBM. |

# V: Fine-tuning and Adaptation

# Training language models to follow instructions with human feedback

**Long Ouyang*** **Jeff Wu*** **Xu Jiang*** **Diogo Almeida*** **Carroll L. Wainwright***

**Pamela Mishkin*** **Chong Zhang** **Sandhini Agarwal** **Katarina Slama** **Alex Ray**

**John Schulman** **Jacob Hilton** **Fraser Kelton** **Luke Miller** **Maddie Simens**

**Amanda Askell†** **Peter Welinder** **Paul Christiano*†**

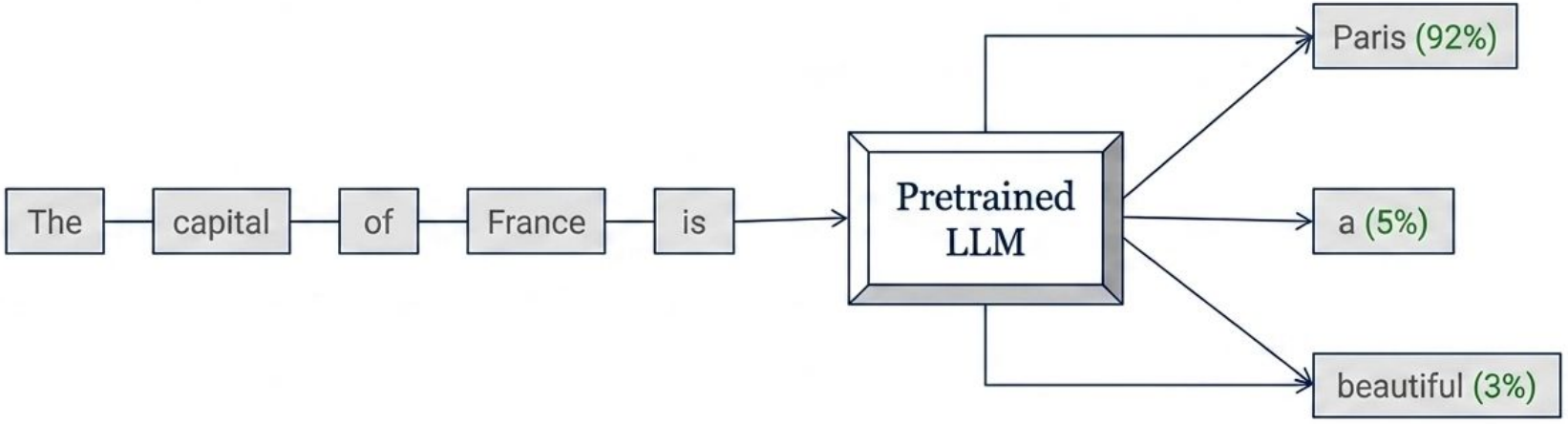
**Jan Leike*** **Ryan Lowe***

OpenAI

## Abstract

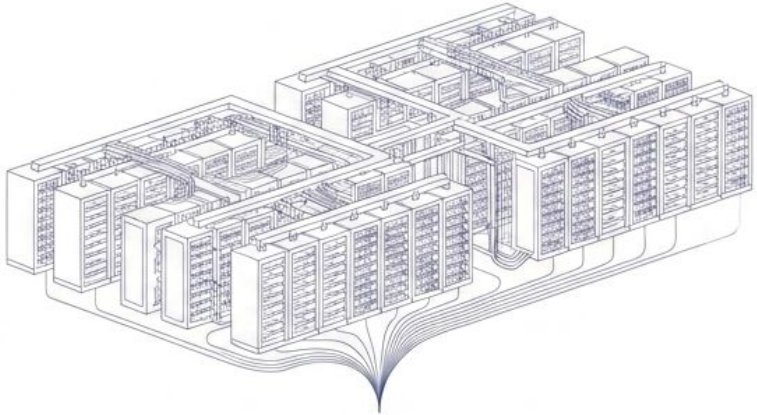
Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for

# A base model operates exclusively as a next-word prediction engine



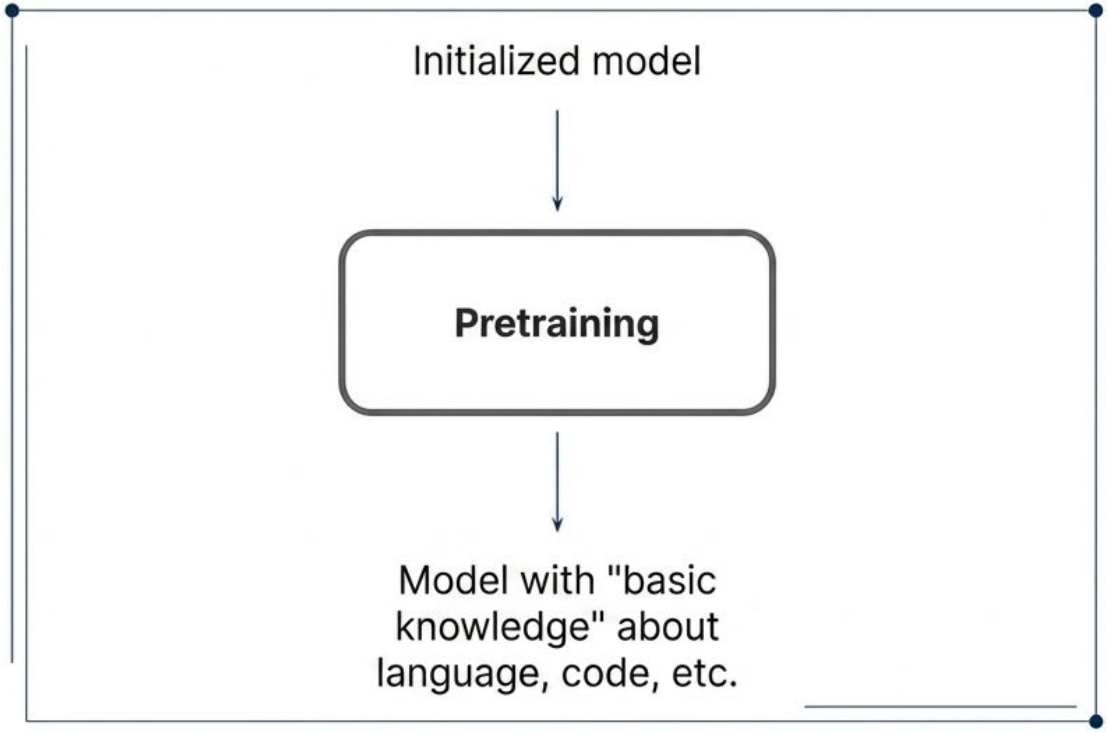

The LLM has been trained for only next-word prediction till now. It is simply spitting out the next probable words based on its raw training dataset.

# Does feeding massive amounts of raw web data make an LLM instantly helpful to users?



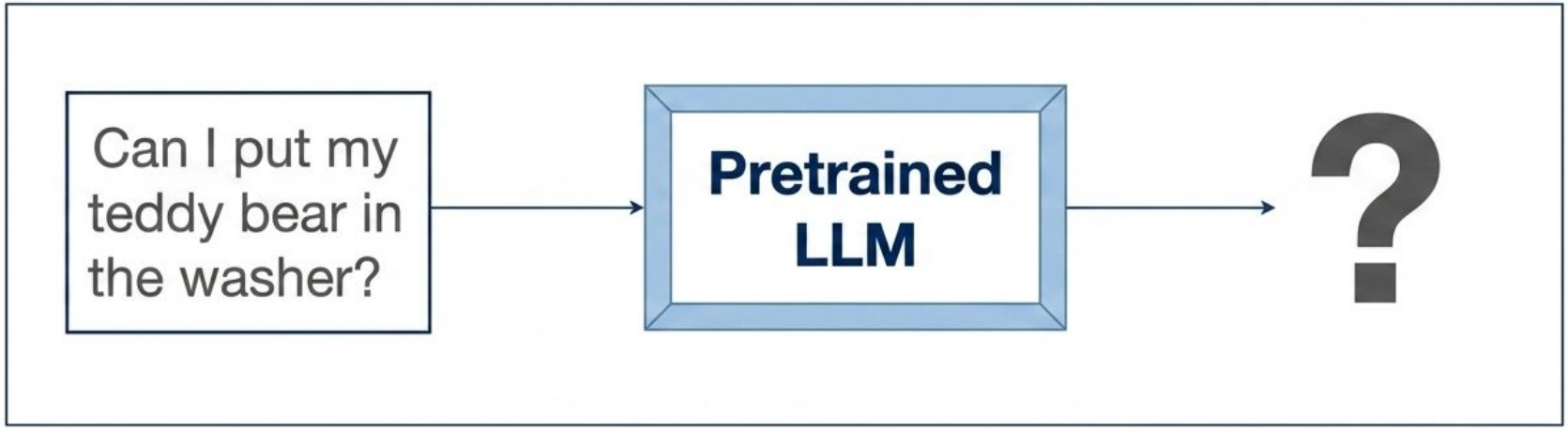
?

# Raw pretraining builds general linguistic knowledge but lacks user-centric direction

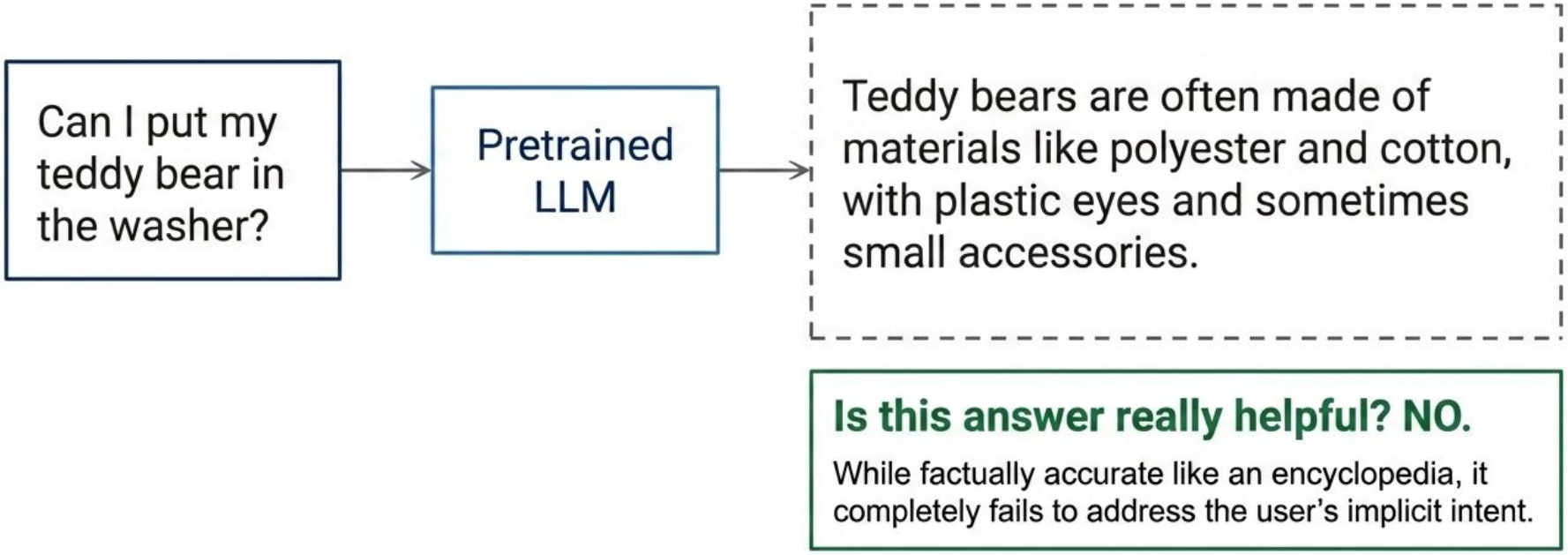



- Requires massive volumes of raw, unstructured data.
- Builds foundational language understanding.
- Core limitation: This general understanding is not yet directly useful to a human user.

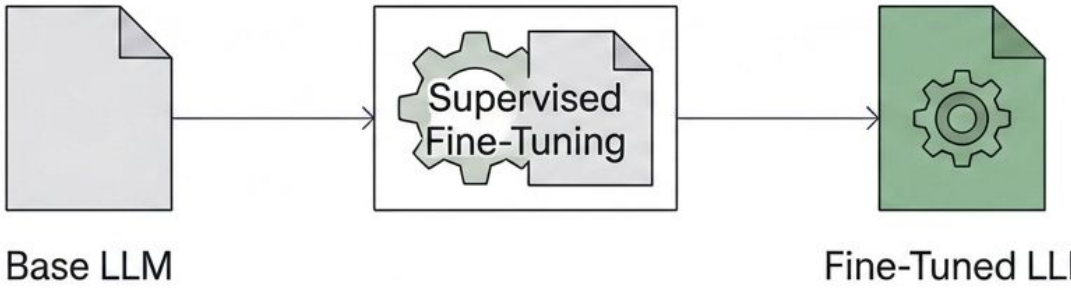
# Testing the utility of a base model with a practical, household question



# Base models prioritize statistical text completion over practical helpfulness



# Supervised Fine-Tuning (SFT) adapts the model's internal weights to specific tasks.




The Engine:

SFT uses precise input/output pairs (supervised data) to enforce desired behaviors.

The Goal:

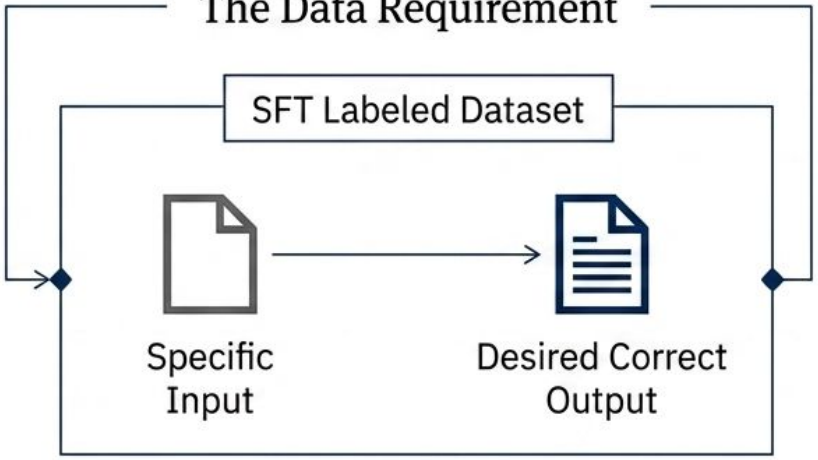
Instead of predicting ANY next word, it trains the model to predict the CORRECT next word given a highly specific input.

# Supervised Fine-Tuning molds model behavior through strict input-output pairs

## The Mechanism

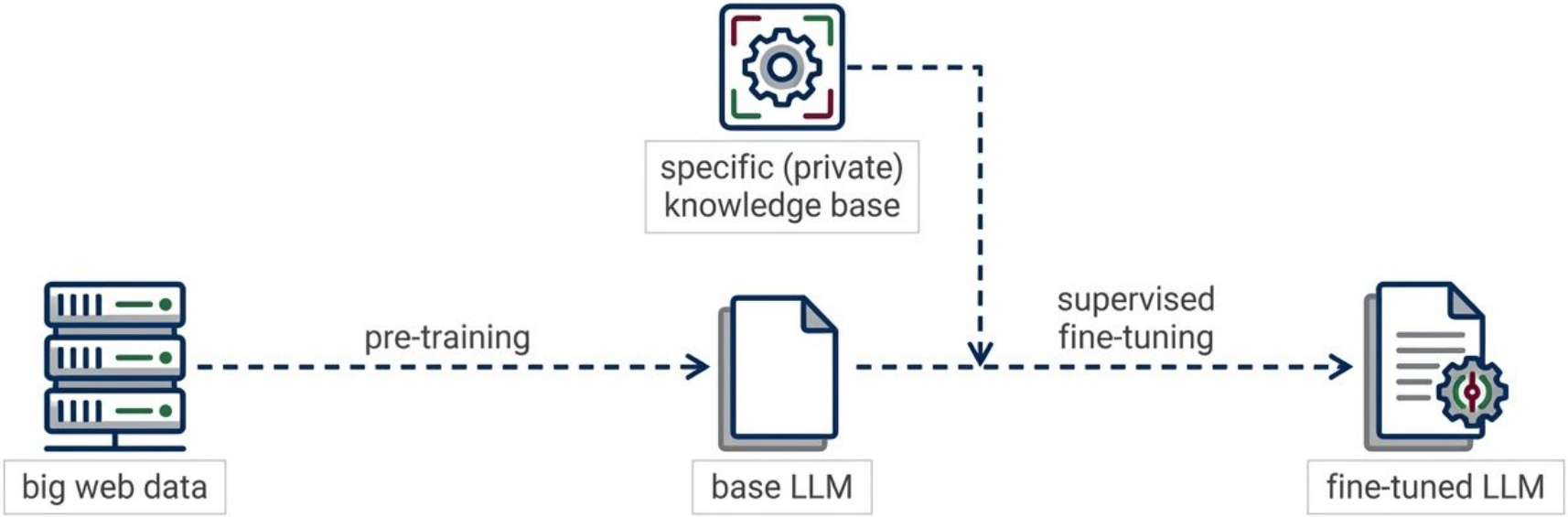
- Supervised Fine-Tuning (SFT) adapts model behavior by actively tuning internal weights.
- SFT relies on providing the model with strict examples of the desired behavior.

## The Data Requirement



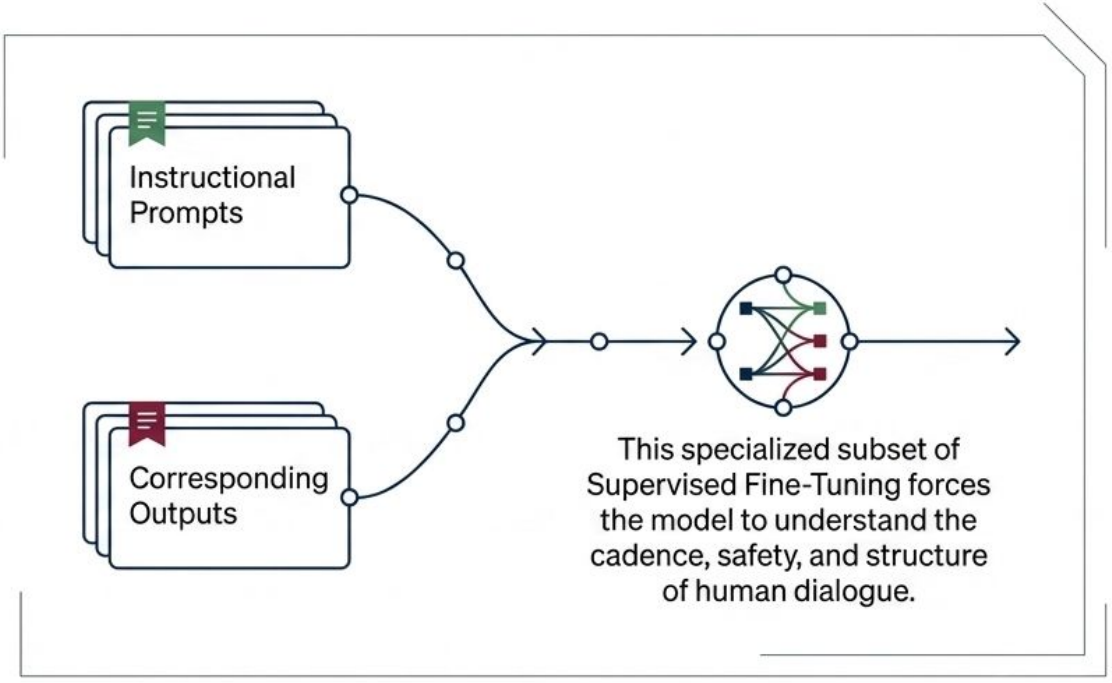

**Goal** → Train the next word prediction behavior specifically given the context of the input prompt.

# The architectural journey from raw web data to a specialized agent



# Instruction tuning bridges the gap between factual recall and conversational AI

Standard pretrained LLMs are inherently unoptimized for conversation or instruction following.



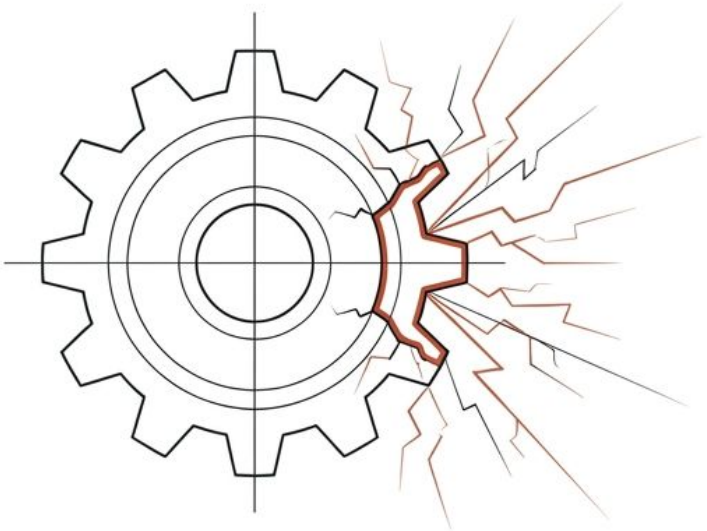
# We apply Instruction Tuning: a specialized form of SFT optimized for conversations.


Key Insight: Instruction tuning requires highly specialized, perfectly labeled datasets of instructional prompts paired with flawless corresponding outputs.

# Comparing the structural shift from baseline prediction to tuned intelligence

| | Base Pretrained LLM | Instruction-Tuned LLM |
|---|---|---|
| Primary Objective | Next-word prediction from raw text. | Fulfilling user intent and following commands. |
| Training Data | Massive, uncurated web scraping. | Small, highly curated input-output conversational pairs. |
| Output Characteristic | Open-ended, encyclopedic, probabilistic. | Direct, bounded, actionable, and safe. |

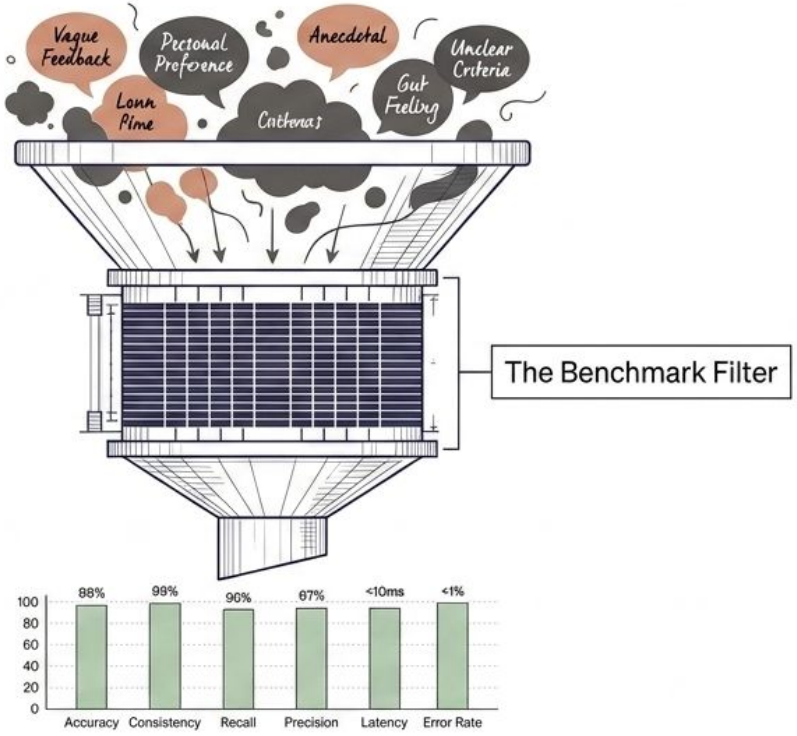
# If Instruction Tuning guarantees alignment, why is deploying a reliable LLM still so difficult?



# Five structural challenges constraining the fine-tuning pipeline

| | | |
|---|---|---|
| 1 | **Human Annotation** | Exceedingly expensive and time-consuming to generate high-quality pairs |
| 2 | **Prompt Distribution** | Models become highly sensitive to how prompts are formatted. |
| 3 | **Generalization Risk** | SFT models are prone to overfitting to their specific training data. |
| 4 | **Evaluation Friction** | Outputs are open-ended, subjective, and often have multiple correct answers. |
| 5 | **Compute Burden** | The process remains computationally expensive to execute at scale. |

# We abandon subjective evaluation in favor of standardized, rigorous benchmark datasets.



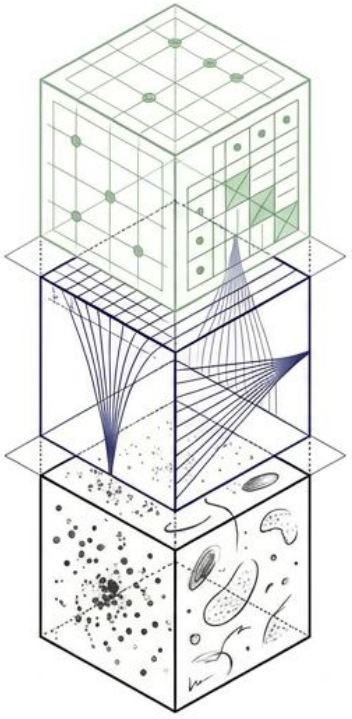

| Core Measurement Vectors | |
|---|---|
| **Knowledge:** Does the model know the core facts? | |
| **Reasoning:** Can it logically connect those facts? | |
| **Problem Solving:** Can it execute algorithmic solutions based on the facts? | |

# The industry relies on a matrix of specialized diagnostic benchmarks.

| Benchmark Name | Core Capability | Source Material | Cognitive Requirement |
|---|---|---|---|
| MMLU | General Knowledge | 50+ subjects (physics, history, etc.) | Broad factual recall. |
| ARC-Challenge | Basic Reasoning | Science questions from school exams | Logical thinking, rejecting mere memorization. |
| GSM8K | Math Reasoning | School-level math dataset | Multi-step logic execution. |
| HumanEval | Code Generation | Python programming problems | Syntax accuracy and algorithmic problem solving. |

# True capability demands a complete, three-phase integrated ecosystem.

Alignment Pathway



Phase 3: Benchmarking (The Verification)
- Objective proof of reasoning & logic.

Phase 2: Instruction Tuning (The Alignment)
- User-centric adaptation via SFT.

Phase 1: Pretraining (The Base)
- Raw knowledge & next-word prediction.

Data grants potential. Tuning grants purpose. Benchmarks grant proof.

Lora: Low-rank adaptation of large lan

# VI. Parameter-Efficient Fine-tuning (LoRA)

# LoRA: Low-Rank Adaptation of Large Language Models

**Edward Hu*** **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
**Yuanzhi Li** **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

## Abstract

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which retrains all model parameters

## 1 Introduction

Many applications in natural language processing rely on adapting *one* large-scale, pre-trained language model to *multiple* downstream applications. Such adaptation is usually done via *fine-tuning*, which updates all the parameters of the pre-trained model. The major downside of fine-tuning is that the new model contains as many parameters as in the original model. As larger models are trained every few months, this changes from a mere "inconvenience" for GPT-2 (Radford et al., b) or RoBERTa large (Liu et al., 2019) to a critical deployment challenge for GPT-3 (Brown et al., 2020) with 175 billion trainable parameters.[1]

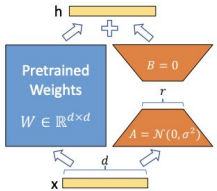


Figure 1: Our reparametrization. We only train $A$ and $B$.

Many sought to mitigate this by adapting only some parameters or learning external modules for new tasks. This way, we only need to store and load a small number of task-specific parameters in addition to the pre-trained model for each task, greatly boosting the operational efficiency when deployed. However, existing techniques

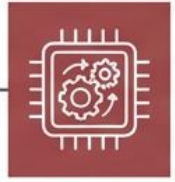
# The Brute-Force Barrier of Supervised Fine-Tuning (SFT)

Traditional SFT updates every parameter in parameter in a pre-trained model. As base base models scale to billions of parameters, this paradigm becomes structurally unsustainable.



## Memory Heavy

Tracking massive weight matrices requires enormous optimizer states and gradients.



## Compute Expensive

Forward and backward passes through the full dense architecture drain processing cycles.

## Hardware Constrained

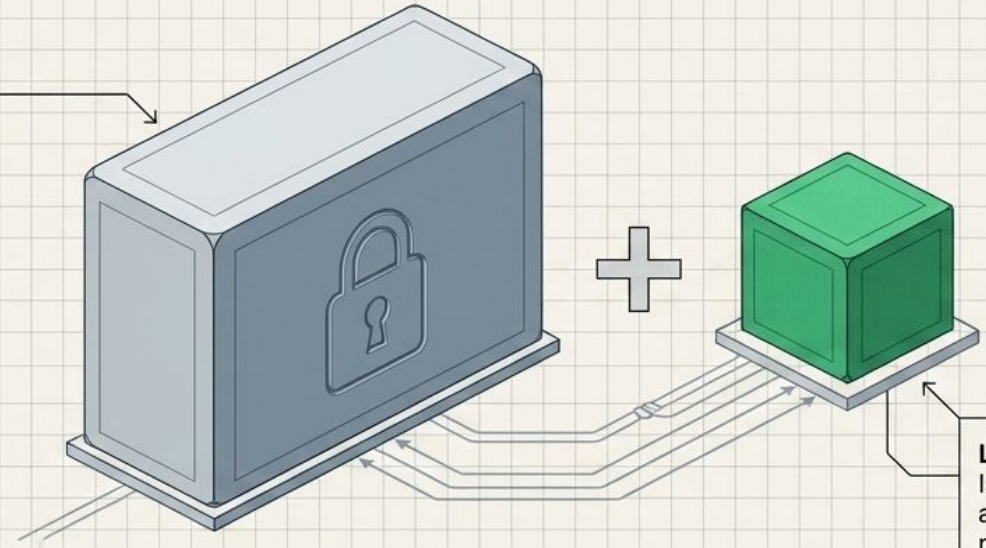
Requires clusters of large-capacity GPUs, creating a hard barrier to entry.

# Can we teach an old model new tricks without rewriting its entire brain?

# Isolate the Base.
# Learn the Correction.

**Keep $W_0$ frozen:** The original pretrained knowledge is locked, requiring zero backpropagation memory.

Frozen Base Matrix ($W_0$)



Correction ($\Delta W$)

**Learn a small correction:** Isolate all new learning into a separate, modular delta matrix.

Instead of changing the full weight matrix **W**, we split the architecture.

# If the correction matrix ($\Delta W$) must match the massive size of the original network, how have we saved any memory?

# Mathematical Foundations: The Low-Rank Property

**The Theorem**

A matrix $\boldsymbol{A}$ of size $m \times n$ is considered **low-rank** if its **rank** $r$ satisfies:

$$\text{rank}(\boldsymbol{A}) \ll \min(m, n).$$

## Translation to Plain English

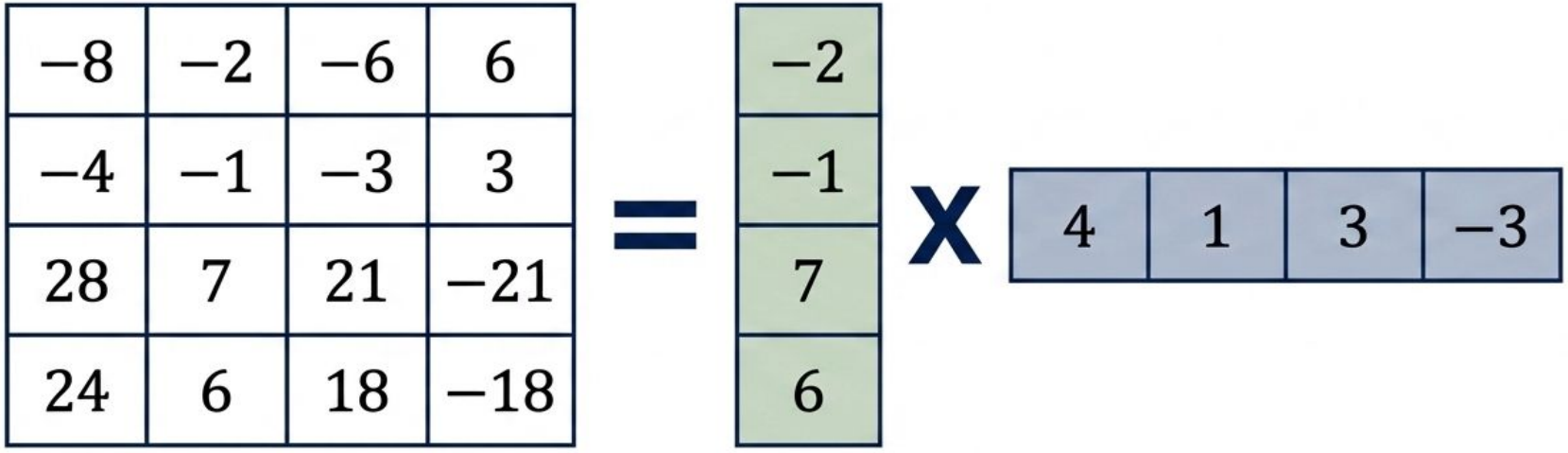
The “rank” dictates the number of linearly independent rows or columns—the true information dimension of a matrix.

A low-rank matrix contains immense mathematical redundancy.

The Core Hypothesis: While LLM weight matrices have full rank, the specific changes required to learn a new task have a very low intrinsic rank. The updates can be drastically compressed without losing fidelity.

# Deconstructing the Matrix: A Numerical Proof

One high-rank matrix represented as a multiple of two low-rank matrices.




| Analytic Takeaway | | |
|---|---|---|
| Original Space:<br>16 parameters to learn<br>(4 × 4). | LoRA Space:<br>8 parameters to learn<br>(4 + 4). | Synthesis: A 50% parameter reduction in this micro-example. At LLM scale (e.g., 10,000 × 10,000), the reduction factor becomes exponential. |

# The LoRA Architecture in Practice

$$W = W_0 + \Delta W \rightarrow W = W_0 + AB$$


$W$ $W_0$ $B$ $A$

### $W_0$

The Pretrained Foundation. Massive, frozen, requires zero gradient updates.

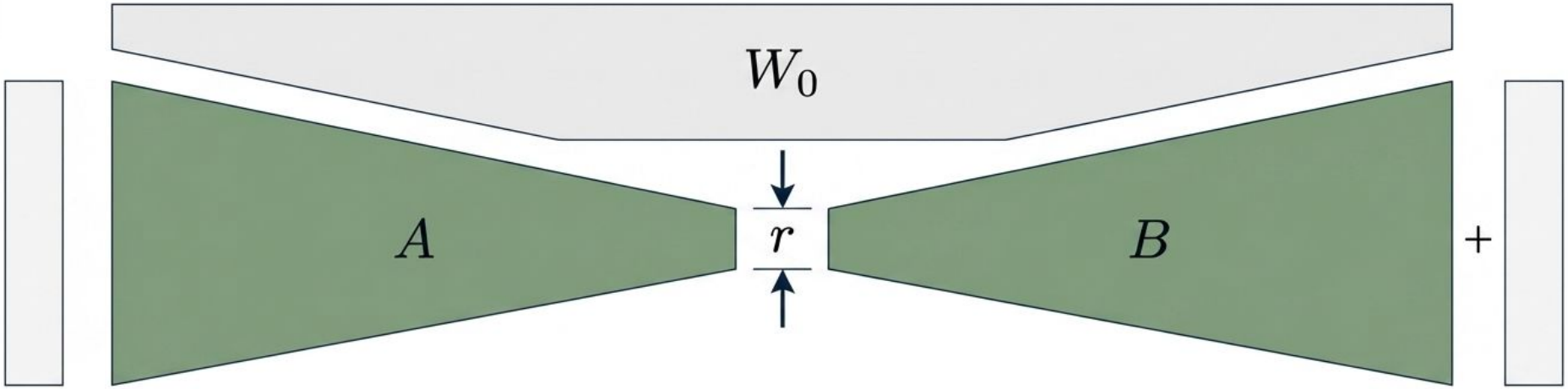
### Matrices $B$ & $A$

The Learnable Adapters. **A** projects dimensions down to a low rank; **B** projects them back up.

### $W$

The Virtual Result. The model operates as if it has a fully dense updated matrix.

# The Hourglass Constraint: Dimension Reduction




## Key Concept: The Rank Hyperparameter ($r$)

- The hyperparameter $r$ controls the width of the mathematical bottleneck.
- Constraint vs Capacity: A smaller $r$ drastically cuts memory/compute by reducing trainable parameters. A larger $r$ increases expressive capacity but at a higher computational cost.
- Design Philosophy: Squeezing updates through $r$ forces the network to isolate and learn only the highest-impact features necessary for the specific downstream task.

# System States: Fine-Tuning vs. Inference

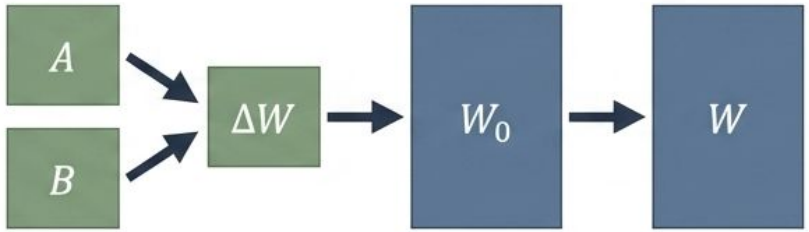
- During finetuning:
  - We learn $W = W_0 + \Delta W$
- In LoRA, instead of learning a huge $\Delta W$ matrix, we learn:
  - Two small matrix $A$ and $B$.
  - $\Delta W = AB$

### During Fine-Tuning (The Update Phase)


- **State**: $W_0$ is completely locked (frozen).
- **Action**: Only the tiny $A$ and $B$ matrices receive gradient updates ($\Delta W = AB$).
- **Advantage**: Massive reduction in GPU memory footprint. Optimizer states are tracked for millions of parameters, not billions.


- During finetuning:
  - We learn $W = W_0 + \Delta W$
- In LoRA, instead of learning a huge $\Delta W$ matrix, we learn:
  - Two small matrix $A$ and $B$.
  - $\Delta W = AB$


### During Inference (The Deployment Phase)




- **State**: $A$ and $B$ are multiplied to form $\Delta W$.
- **Action**: $\Delta W$ is mathematically merged into $W_0$ to create the final static matrix $W$ ($W = W_0 + AB$).
- **Advantage**: Zero inference latency penalty. The model runs exactly as fast as the baseline dense architecture.

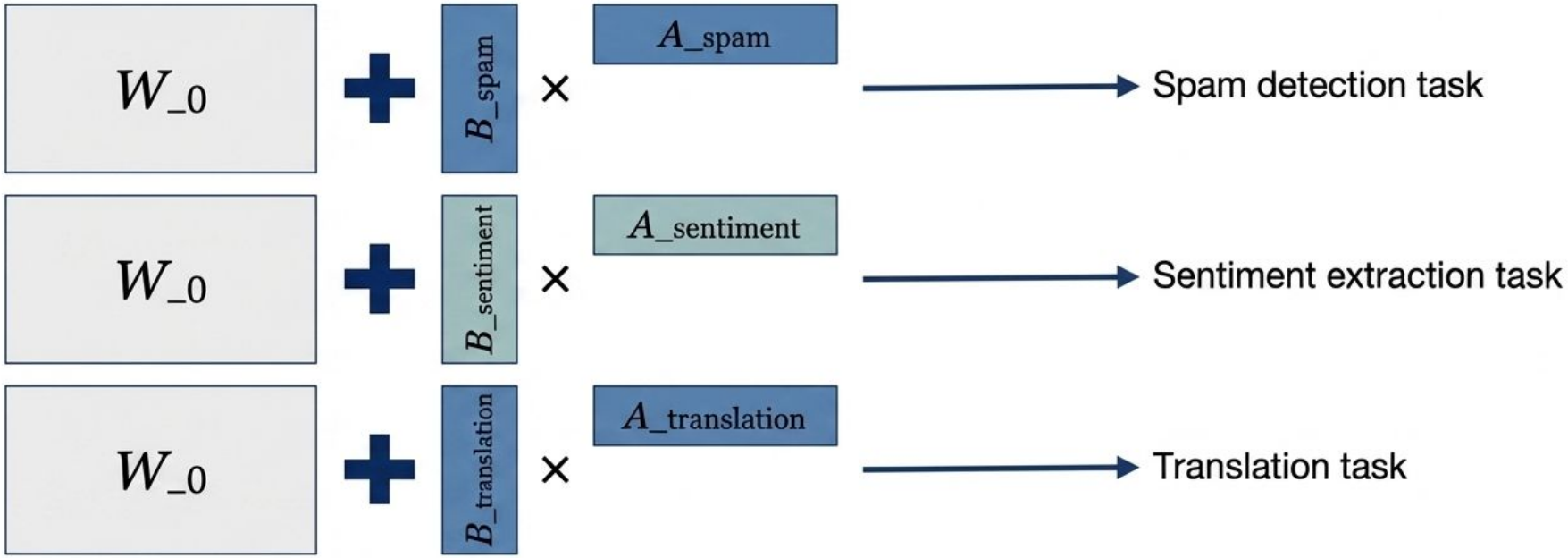
# Architectural Comparison: Standard SFT vs. LoRA

| Dimension | Standard Supervised FT | LoRA (Low-Rank Adaptation) |
|---|---|---|
| Trainable Parameters | 100% (Billions) | $\ll$ 1% (Millions) |
| Memory Requirement | Massive (Requires multiple/large GPUs) | Minimal (Runs on single consumer GPUs) |
| Inference Latency | Unchanged (Baseline) | Unchanged (Zero penalty after weight merge) |
| Storage per Task | Massive (Full 50GB+ copy of the LLM) | Tiny (10MB - 50MB for matrices $A$ & $B$) |

**Core Insight:** LoRA entirely decouples the size of the base model from the size of the task-specific updates, democratizing LLM adaptation.

# Operational Agility: Swap Matrices = Swap Tasks

Because $W_0$ remains pristine, transitioning a model to a new capability simply requires swapping the tiny $A$ and $B$ matrices.



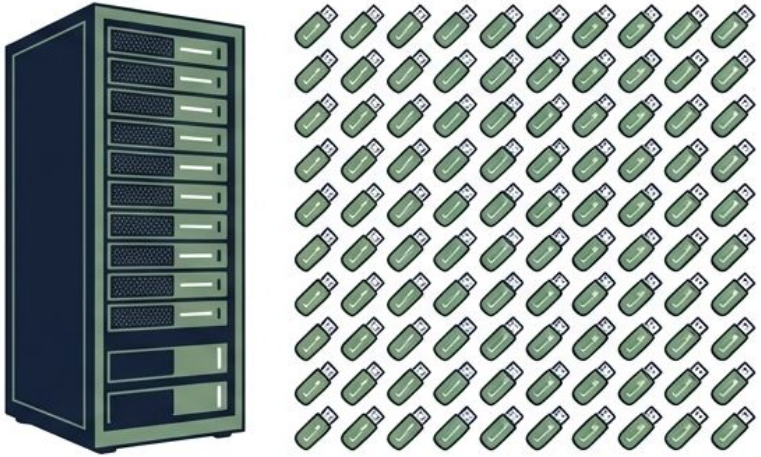

**System Advantage:** Host the massive base model in VRAM only once. Serve multiple fine-tuned applications simultaneously by dynamically applying lightweight adapters per user request.

# The Economics of Storage and Deployment

## Traditional SFT Scale

Deploying for 100 enterprise customers requires storing **100 complete copies** of a **50GB+** model (**5 Terabytes** of storage).

## The LoRA Scale



- **One Base Model:** Store the 50GB W_0 model exactly **once** in the cloud environment.
- **100 Lightweight Adapters:** Store **100 specific LoRA configurations** (Matrices A & B), each ranging from 10MB to 50MB.

**The Bottom Line:** LoRA transforms specialized AI infrastructure from prohibitive multi-tenant server cloning to highly scalable, megabyte-scale micro-deployment.

# Empirical Observations & Training Constraints

## Constraint 1: Learning Rate Dynamics

**Finding:** LoRA requires a higher learning rate compared to full fine-tuning.

**Mechanism:** Because the parameter space is drastically reduced (millions instead of billions), gradients apply to a much smaller set of weights. A larger step size is required to push these few parameters enough to meaningfully impact the final output.

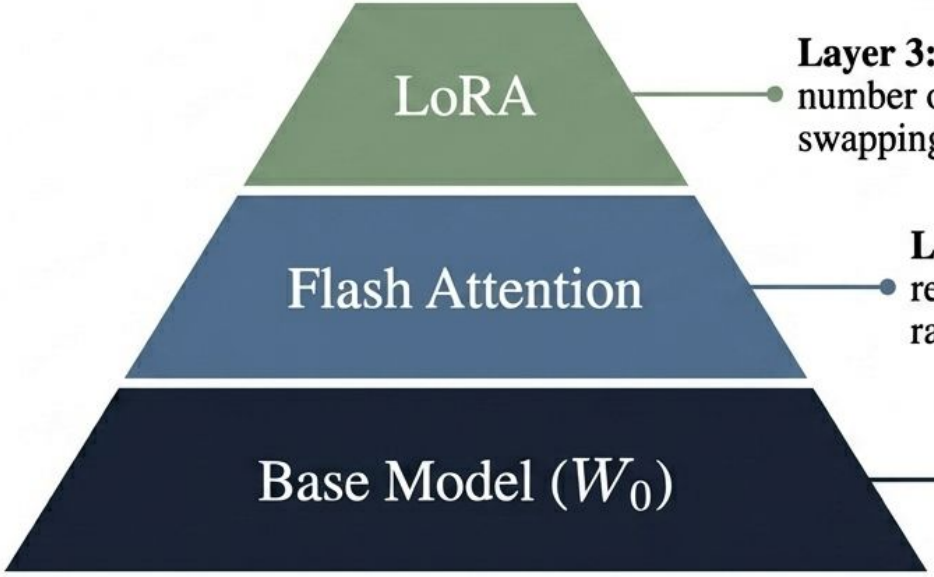
## Constraint 2: Batch Size Sensitivity

**Finding:** LoRA performs poorly on large batch sizes compared to full fine-tuning.

**Mechanism:** The limited expressive capacity (the bottleneck $r$) struggles to capture and resolve the highly diverse, blended gradient signals that arise when averaging across massive batches of data.

# The Modern Efficient Adaptation Stack

Syntheis: LoRA does not exist in a vacuum; it acts as the top layer of a compounded efficiency ecosystem.




**Layer 3: Parameter Efficiency.** Minimizes the number of parameters updated. Enables modular task swapping and drastically cuts storage.

**Layer 2: Compute Efficiency.** Minimizes memory reads/writes during the attention calculation, radically speeding up raw processing time.

**Layer 1: Pre-Trained Foundation.** Remains completely frozen, preserving general intelligence without computational degradation.

**Conclusion:** Together, this ecosystem stack enables state-of-the-art fine-tuning on accessible, consumer-grade hardware.

Distilling the Knowledge in a Neural Network

# VII. Knowledge Distillation

# Distilling the Knowledge in a Neural Network

**Geoffrey Hinton**[*†]
Google Inc.
Mountain View
geoffhinton@google.com

**Oriol Vinyals**[†]
Google Inc.
Mountain View
vinyals@google.com

**Jeff Dean**
Google Inc.
Mountain View
jeff@google.com

## Abstract

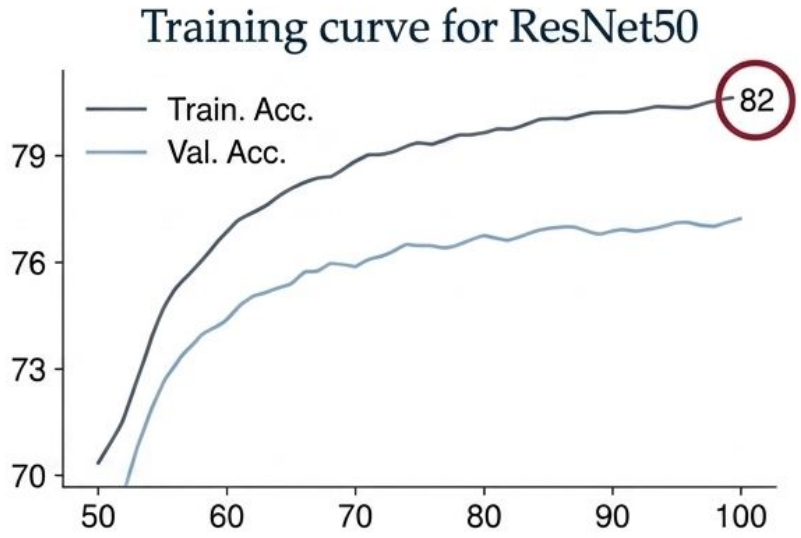
A very simple way to improve the performance of almost any machine learning algorithm is to train many different models on the same data and then to average their predictions [3]. Unfortunately, making predictions using a whole ensemble of models is cumbersome and may be too computationally expensive to allow deployment to a large number of users, especially if the individual models are large neural nets. Caruana and his collaborators [1] have shown that it is possible to compress the knowledge in an ensemble into a single model which is much easier to deploy and we develop this approach further using a different compression technique. We achieve some surprising results on MNIST and we show that we can significantly improve the acoustic model of a heavily used commercial system by distilling the knowledge in an ensemble of models into a single model. We also

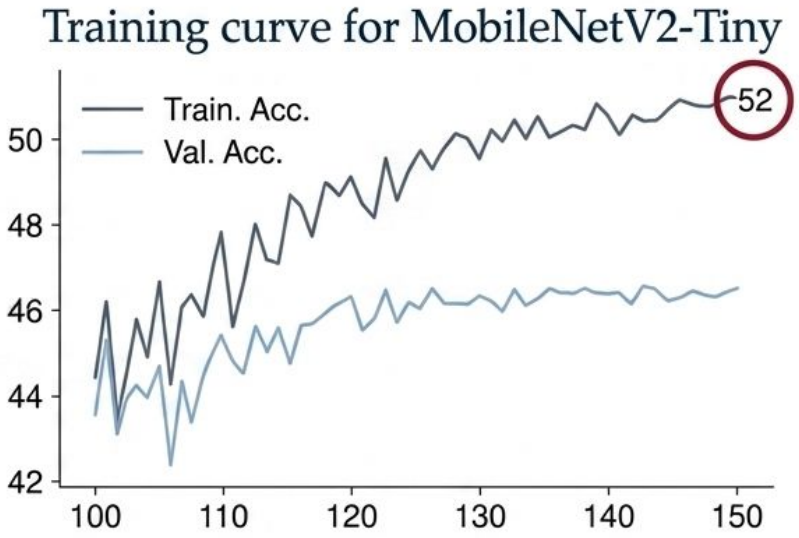
# Why can’t we simply deploy our best models to to the edge?

| | Cloud AI | Tiny AI (Edge) |
|---|---|---|
| |   |   |
| Computation (fp32) | **19.5 TFLOPS** | **MFLOPs** |
| Memory | **80GB** | **256kB** |
| Neural Network | **ResNet, ViT-Large** | **MCUNet, MobileNetV2-Tiny** |

**High latency, high memory usage, and expensive inference render cloud-scale models wholly unsuitable for mobile, edge, and real-time systems.**

If hardware is the limit, why not just train tiny models from scratch?



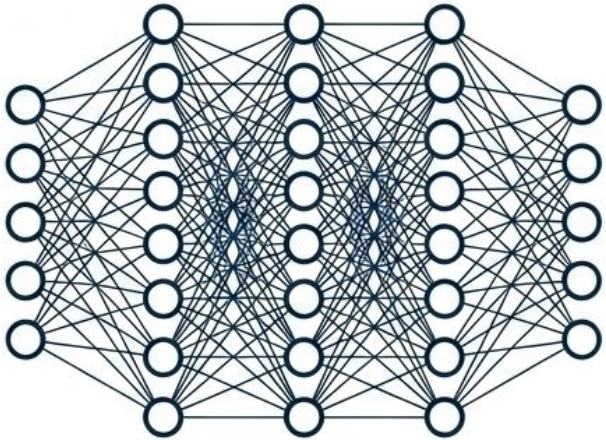

## The Hardware/Training Paradox.

- **The Bottleneck:** Neural networks must be drastically compressed to run efficiently on edge hardware.
- **The Reality:** Tiny models are prone to severe underfitting on large, complex datasets. They lack the capacity to learn intricate patterns from scratch.

**Can we help the training of tiny models by leveraging large models?**

# The Concept of Knowledge Distillation

## The Teacher

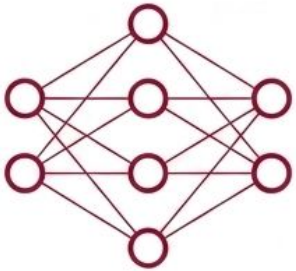


Large Model / Ensemble

High-capacity brain, highly accurate, too computationally heavy for deployment.

Knowledge Transfer

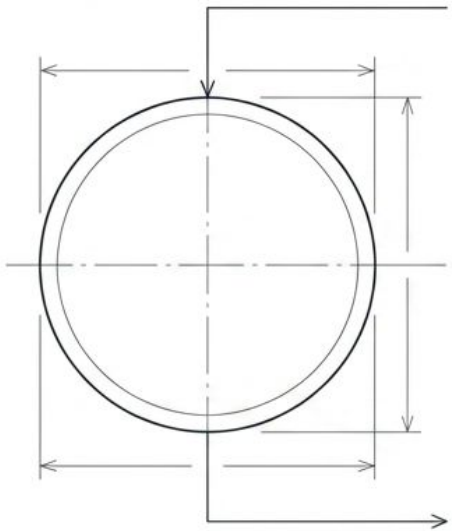
## The Student



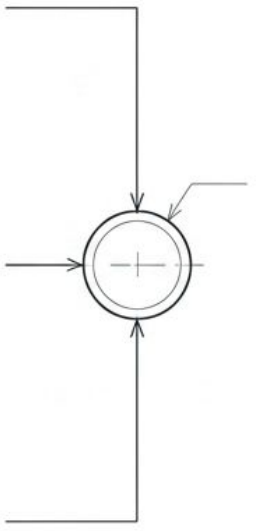
Tiny Model

Limited capacity, highly efficient, struggles to learn complex patterns autonomously.

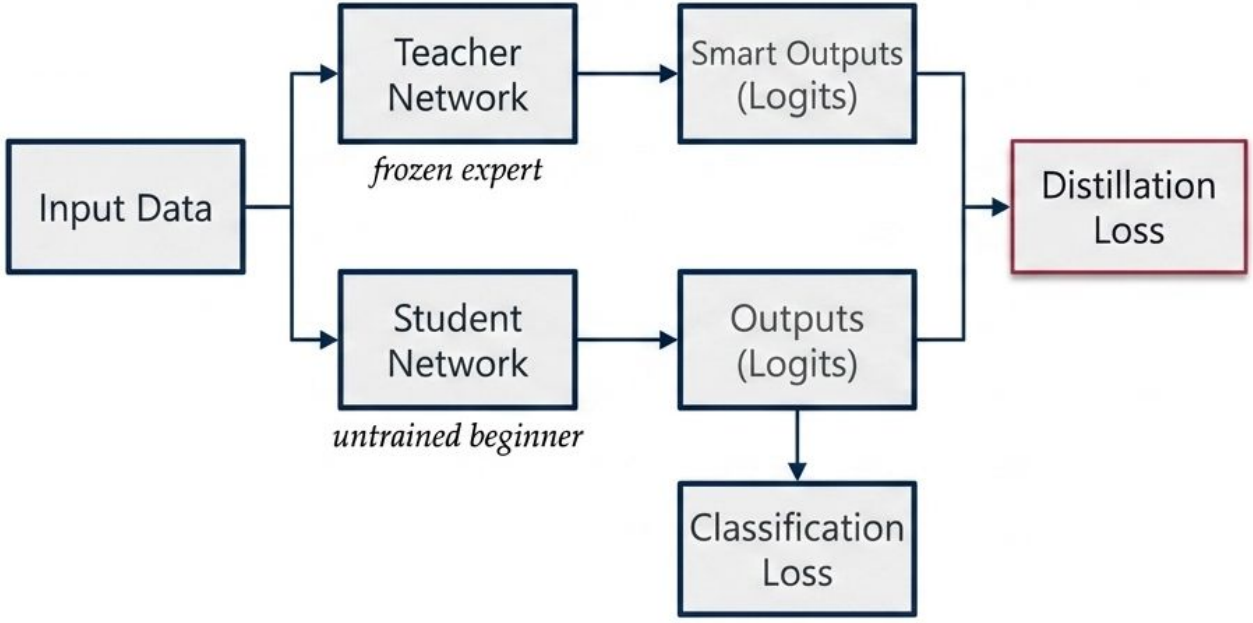
Hinton, Vinyals, Dean (2015). Distilling the Knowledge in a Neural Network.



# How do we get the high capacity of a large brain into the limited footprint of a small one?



# Knowledge Distillation: Architectural Framework




## The Core Insight

The student is trained not just to guess the right answer, but to think like the expert. It must match the Teacher's raw, pre-softmax logits.

# The Dual Learning Objective

## Classification Loss (Standard Learning)

$$\mathbb{E}(-p_t \log p_s)$$

Normal cross-entropy loss against ground-truth labels. The student learns what the actual object is.

## Distillation Loss (Mimicry)

$$\mathbb{E}(||\mathbf{p}_t - \mathbf{p}_s||_2^2)$$

L2 Loss (or KL Divergence) matching the student's logits directly to the teacher's logits.

**Total Loss = Classification Loss + Distillation Loss**

# Does the student just memorize the teacher’s final answers?

## Standard Softmax Prediction

| | |
|---|---|
| Cat | -> 0.999 |
| Dog | -> 0.0008 |
| Car | -> 0.0001 |
| Airplane | -> 0.0001 |

## The Illusion of Perfection

- The prediction looks flawless, but it suffers from Information Collapse.
- Because the model is highly confident, all non-target classes are squashed to near-zero.
- Visually and structurally, a Cat is closer to a Dog than to an Airplane, but this standard probability distribution entirely erases that relationship.

# Defining Dark Knowledge

Dark Knowledge refers to the hidden structure of relative probabilities across all classes, not just the correct one.

**Teacher Distribution**

| | Logits | Probabilities |
|---|---|---|
| Cat | 5 | 0.982 |
| Dog | 1 | 0.017 |

**Student Target Learning**

| | Logits | Probabilities |
|---|---|---|
| Cat | 3 | 0.731 |
| Dog | 2 | 0.269 |

**Nuanced belief distribution:**

1. Cat is highly likely.
2. Dog is somewhat likely. (Relative similarity)
3. Car is less likely.
4. Airplane is almost impossible.

The student's primary task is to map and replicate the teacher's relative incorrectness.

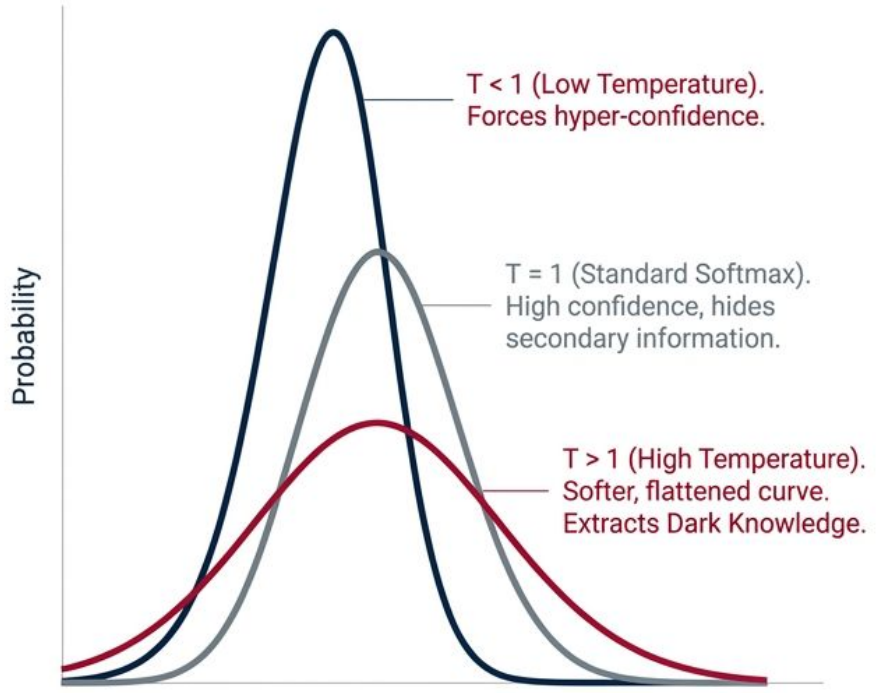
If this crucial knowledge is hidden behehind near-zero probabilities,

how do we force the student to see it?

# The Mechanism: Softmax with Temperature (T)

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

zi = raw logits



Original Logits:

$$[10, 2, 0, -1]$$

Normal Softmax ($T$=1):

$$[0.999, 0.0008, \text{near zero} \ldots]$$

— Information is collapsed.

Applied Temperature ($T$=5):

$$[\frac{10}{5}, \frac{2}{5}, 0, -\frac{1}{5}\}] = [\mathbf{0.60}, \mathbf{0.20}, \mathbf{0.12}, \mathbf{0.08}]$$

The Revelation: Under T=5, the hidden structure emerges clearly. Dog (0.20) is distinct from Car (0.12) and Plane (0.08). Temperature does not distort the truth; it acts as a magnifying glass for hidden relational structure.

# Is matching the final probability distribution the only way to transfer knowledge?

# The Deep Distillation Taxonomy

*What else can be distilled beyond final outputs?*

## Category 1: Termini

- Output Logits (What we've covered)

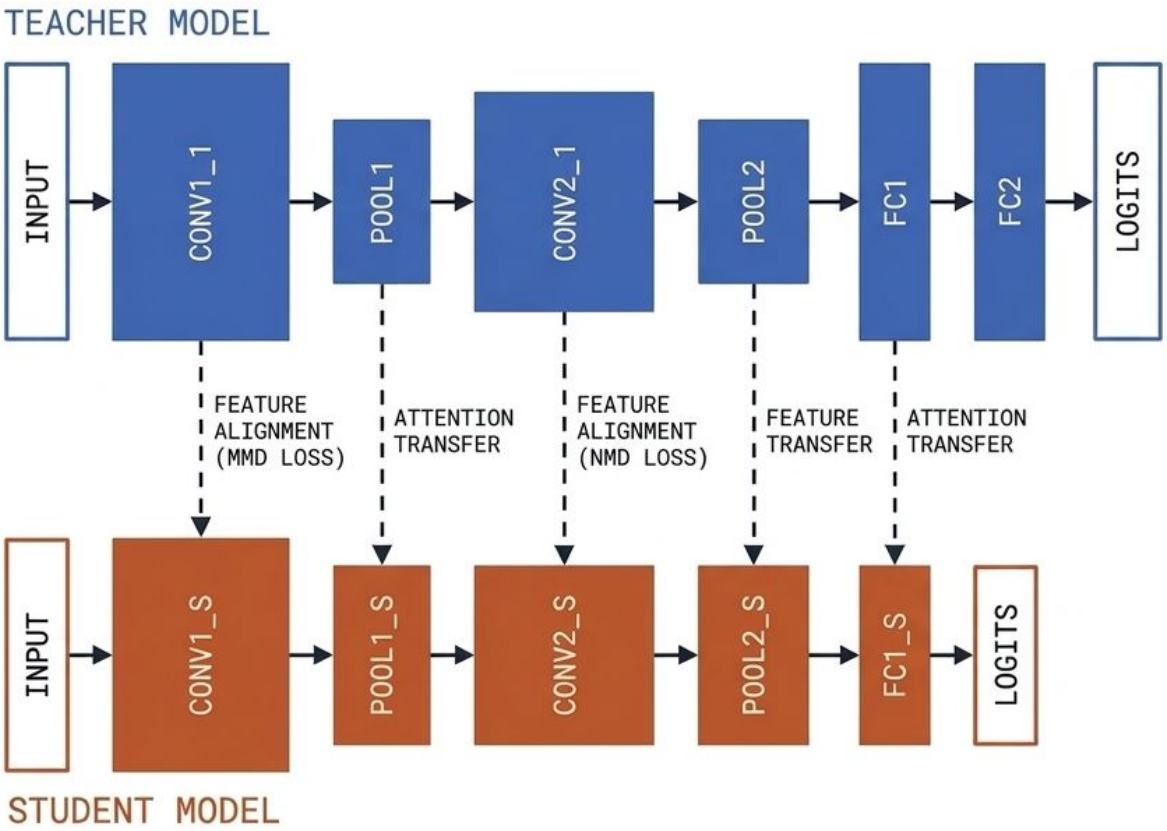
## Category 2: Internals

- Intermediate Weights
- Intermediate Features

## Category 3: Mechanisms

- Gradients (Attention)
- Sparsity Patterns (Activation)
- Relational Information

# WE CAN FORCE THE STUDENT TO LEARN THE TEACHER'S EXACT REASONING PATHWAYS.




TEACHER ATTENTION MAP
(WHERE IT LOOKS)

## WHAT CAN BE DISTILLED?

**INTERMEDIATE FEATURES**

Learning a similar representation space (MMD Loss).

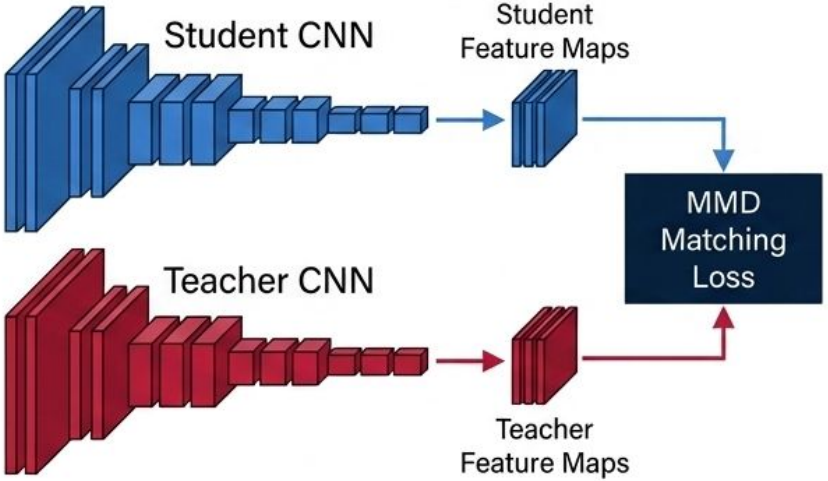
**GRADIENTS**

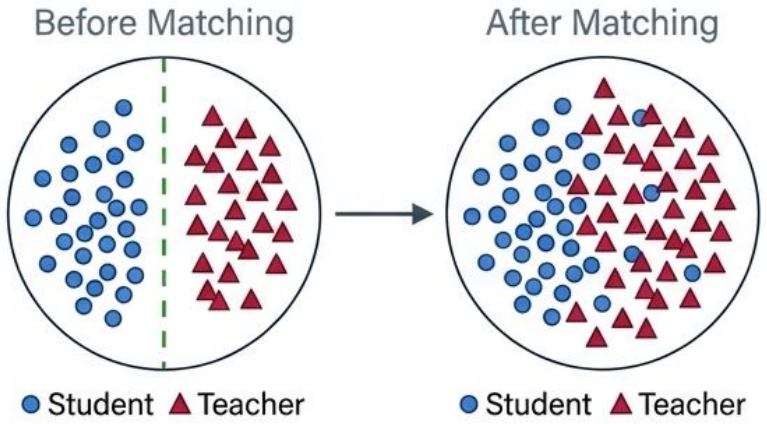
Mimicking the attention map to learn where to look.

**SPARSITY PATTERNS**

Replicating ReLU ON/OFF neuron states.

# Intermediate Feature Matching: Distilling Internal Representations

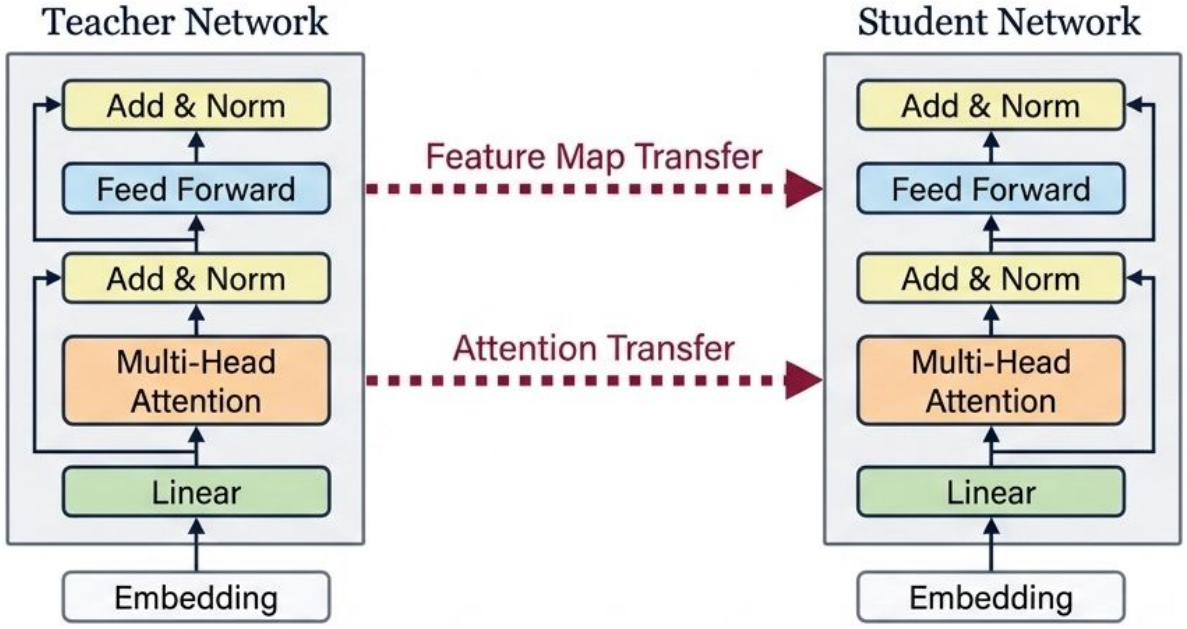



**Deep Distillation:** We match the distributions of feature maps, not necessarily their exact numerical values. This relies on MMD (Maximum Mean Discrepancy), which measures and minimizes the distance between the two feature distributions, forcing the student to learn the internal reasoning pathways of the teacher.

# Does this blueprint hold up for modern architectures like Transformers?

# State-of-the-Art: Knowledge Distillation in Transformers




Modern LLMs and Vision Transformers rely heavily on KD to function on edge devices. Knowledge Distillation has evolved from simply softening output probabilities to wholesale architectural mimicry, serving as the critical bridge between cloud-scale capability and edge-scale deployment.